

obtain_local_deps

Revision: 1 **Last modified:** 2026-06-28T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** §11.4.18 script companion doc for the per-host local-dependency obtaining mechanism (source: scripts/obtain_local_deps.sh)

Overview

scripts/obtain_local_deps.sh is the project's cross-platform LOCAL-DEPENDENCY obtaining mechanism: for every host we distribute to, it makes each runtime dependency the host may be missing available **locally and git-ignored**, out-of-the-box during setup.sh. The first (and currently only) consumer is **jemalloc** — the project's core hardening allocator — which the tmux binary links **dynamically** (DT_NEEDED libjemalloc.so.2 on Linux / LC_LOAD_DYLIB on macOS). The binary must therefore find a real libjemalloc.so.2 (or libjemalloc.2.dylib) at runtime, and two real hosts cannot:

- **amber** — has NO system jemalloc AND no sudo to install one. A container-built ELF carrying DT_NEEDED libjemalloc.so.2 simply cannot start (libjemalloc.so.2: cannot open shared object file). The fix is to OBTAIN a host-runnable library into .local-deps/.
- **mistborn** — HAS jemalloc via Homebrew, but command -v brew FAILS under a non-interactive SSH PATH (the setup.sh exit-3 root cause), so setup never resolves the already-present library. The fix is to RESOLVE it by ABSOLUTE path (§11.4.111) — never by ambient command -v / \$PATH.

The script does two things, in this order, per dependency:

1. **RESOLVE** an already-present copy by ABSOLUTE path (§11.4.111).
2. **OBTAIN** it git-ignored into .local-deps/<uname-s>_<uname-m>/ when genuinely missing (§11.4.77) — a source build, or a container extract on Linux as a fallback.

jemalloc **stays dynamic** — this script only makes the shared library AVAILABLE. The binary/wrapper then find it via the patchelf rpath that setup.sh Step 2b stamps onto the ELF + the absolute LD_PRELOAD (DYLD_INSERT_LIBRARIES on macOS) the generated tmx wrapper exports. This composes §11.4.77 (regen mechanism for git-ignored content), §11.4.81 (cross-platform

parity: Linux source/container + macOS source/brew), and §11.4.111 (resolve-by-stable-name, never by enumeration / ambient PATH).

Prerequisites

The script is invoked as `bash` (it uses bash arrays + case), and is `bash -n` clean per §11.4.67. It needs NO `sudo` and mutates no host package state (except an explicit `brew install` fallback on macOS). Per obtain method:

- **Resolve (any host)** — `ldconfig` (Linux) / an absolute `brew` (macOS) / `pkg-config` / `common-lib-dir` globs. No build tools needed when the dependency is already present.
- **Obtain via source** — a C compiler (`cc` / `gcc` / `clang`) + `make`
 - a downloader (`curl` or `wget`) + `tar` + a sha256 tool (`sha256sum` or `shasum`).
- **Obtain via container (Linux fallback)** — `podman` or `docker` + the `docker/Dockerfile` build image (which already ships `libjemalloc-dev`). Building the image needs network for `apt`; extracting from an existing image does not.

Usage

```
# Resolve-or-obtain every default dependency (jemalloc).
bash scripts/obtain_local_deps.sh

# Skip host detection – always obtain into the local prefix.
FORCE_OBTAIN=1 bash scripts/obtain_local_deps.sh

# Override the git-ignored root.
LOCAL_DEPS_ROOT=/path/to/deps bash scripts/obtain_local_deps.sh

# Restrict to a subset of dependency names.
DEPS="jemalloc" bash scripts/obtain_local_deps.sh

# Force the Linux obtain method (auto | source | container).
OBTAIN_METHOD=container bash scripts/obtain_local_deps.sh
```

Inputs (env, all optional):

Var	Default	Meaning
FORCE_OBTAIN	0	1 → skip host detection, obtain into the local prefix.
LOCAL_DEPS_ROOT	<repo>/. <code>local-deps</code>	Override the git-ignored root.
DEPS	<code>jemalloc</code>	Space-separated dependency names.

Var	Default	Meaning
OBTAIN_METHOD	auto	Linux obtain method: auto source container.

Outputs:

- `.local-deps/<uname-s>_<uname-m>/lib/<libname>` — the obtained library (when an OBTAIN was needed).
- `.local-deps/<uname-s>_<uname-m>/resolved.env` — a sourceable KEY=VALUE file: LOCAL_DEPS_PREFIX, plus per-dependency JEMALLOC_S0 (absolute path to the resolved/obtained library), JEMALLOC_LIBDIR (its directory), JEMALLOC_SOURCE (a best-effort provenance label: host-system / host-brew / local-deps / local-build / container-extract).
- `RESOLVED <dep> → <path> (<source>) / OBTAINED <dep> → <path> (<source>)` lines on stdout.

Edge cases

- **Already present (idempotent reuse)** — a present + valid local library, or a host-resolvable copy, is REUSED, not rebuilt. Re-running the script is cheap and side-effect-free.
- **Offline / network unreachable** — a source download that fails exits with typed code **11** (EC_NETWORK); the caller (setup / test) treats this as a §11.4.3 SKIP-with-reason rather than a fake success. A cached tarball whose sha256 already matches is reused without touching the network.
- **sha256 mismatch** — a downloaded tarball whose digest differs from the pinned value is deleted and the script exits **12** (EC_SHA) — never a build from unverified source.
- **No C compiler (Linux)** — auto falls back to the container extract (the amber path); an explicit OBTAIN_METHOD=container forces it. With neither a compiler nor podman/docker, the script exits **10** (EC_NO_TOOLCHAIN).
- **No C compiler (macOS)** — falls back to brew install <dep> via an ABSOLUTE brew (/opt/homebrew/bin/brew then /usr/local/bin/brew), then re-resolves; exits **10** if that fails too.
- **patchelf absent / rpath not applied** — the library is still found via the wrapper's absolute LD_PRELOAD / LD_LIBRARY_PATH (Linux) or DYLD_INSERT_LIBRARIES / DYLD_LIBRARY_PATH (macOS) sourced from resolved.env; the rpath is belt-and-suspenders, not the only path.
- **Unknown dependency / unsupported OS** — a DEPS name not in the registry, or an OS that is neither Linux nor Darwin, exits **14** (EC_UNSUPPORTED) per §11.4.6 (no-guessing — never invent a recipe).

Typed exit codes (never fake success): 0 ok · 10 no obtain toolchain
· 11 network unreachable · 12 sha256 mismatch · 13 container
obtain failed · 14 unsupported dependency/OS.

Internal behaviour

1. **Plat key** — `PLAT="$(uname -s)_$(uname -m)"` scopes the local prefix per OS+arch so a multi-arch checkout never cross-contaminates.
2. **Declarative registry** — `dep_field <dep> <field>` (a bash-3.2-safe case, no associative arrays) holds each dependency's pinned version, release URL, sha256, per-OS library names, pkg-config name, brew name, and env prefix. Add a dependency by adding case branches — never by guessing a recipe at runtime.
3. **RESOLVE order (first hit wins, all ABSOLUTE)** — (1) `pkg-config --variable=libdir` from absolute pkg-config candidates → (2) `ldconfig -p` last field (Linux) → (3) absolute Homebrew prefix (`/opt/homebrew/bin/brew /usr/local/bin/brew` — the mistborn fix) → (4) common absolute lib dirs → (5) a previously-obtained `.local-deps/` copy. The resolver NEVER uses ambient command `-v` for the dependency itself (§11.4.111); the `_first_exe` helper only falls back to a PATH lookup for TOOLCHAIN binaries (compilers/make/curl).
4. **OBTAIN** — `obtain_via_source` downloads the pinned tarball (cache + sha256-verify), `./configure --disable-debug`, then `make build_lib_shared + make install_lib_shared install_include` (shared library only, fast). `obtain_via_container` (Linux fallback) builds / reuses the docker/Dockerfile image and `cp -Ls` the dereferenced `.so` out through a bind-mounted, host-owned / out.
5. **resolved.env** — written atomically (`.tmp` then `mv`) with the per-dependency `*_SO / *_LIBDIR / *_SOURCE` triple that `setup.sh` and the `tmx` wrapper source. The overall exit code is the first non-zero typed code encountered (no fake success).

Related scripts

- `scripts/setup.sh` — invokes this script (Step 1b) and applies the `patchelf rpath` from `resolved.env` (Step 2b).
- `scripts/tmx.template` — the generated wrapper exports the absolute `LD_PRELOAD / DYLD_INSERT_LIBRARIES (+ *_LIBDIR` on the library path) from `resolved.env` so the dynamic `jemalloc` is found at runtime.
- `scripts/tests/67_local_deps.sh` — the runtime / anti-bluff coverage (`jemalloc` stays dynamic; the obtained library is host-runnable) plus its paired §1.1 meta-test mutation.
- `scripts/verify.sh` — the SOURCE-layer CM-LOCAL-DEPS-MECHANISM pre-build gate asserting the mechanism is wired end-to-end.

- `.gitignore-meta/local_deps.yaml` — the §11.4.77 regeneration manifest for the git-ignored `.local-deps/` tree.
- `docs/research/local_deps_20260628/research.md` — the deep-research notes (§11.4.8) behind the resolve-by-absolute-path + obtain design.

Last verified

2026-06-28 — `bash -n scripts/obtain_local_deps.sh clean; resolve / obtain` behaviour validated against the `amber (obtain)` and `mistborn (absolute-path resolve)` host drivers. Runtime captured-evidence is gated on `scripts/tests/67_local_deps.sh`.